B.TECH COMPUTER SCIENCE & ENGINEERING | CAPSTONE RESEARCH PAPER

Design and Implementation of Government Scheme Finder: A Citizen-Centric Web Platform with Multi-Factor Eligibility Filtering and Future RAG Integration

A Full-Stack e-Governance Solution Indexing 4,732 Official Schemes with Zero-Trust Security

Rudra Srivastava
Undergraduate Student, Department of Computer Science & Engineering
Project Repository: *https://github.com/srivastavarudra22-prog/government-scheme-finder.git* • Academic Year 2026

Abstract—In India, central and state governments spend trillions of rupees on welfare schemes for farmers, students, women, and small business owners. However, millions of eligible citizens miss out on these benefits because information is scattered across hundreds of departmental websites, application procedures are complicated, and citizens often encounter broken links or cyber scams. To solve this real-world problem, we developed **Government Scheme Finder**, a responsive, full-stack citizen portal indexing **4,732 official Indian government schemes** across 36 States/UTs and 28 Central Ministries. The system features an instant **8-Factor Eligibility Wizard** (evaluating age, income, state, gender, caste, occupation, education, and residence area), an **Intelligent Official Redirection Engine** that routes users only to verified government portals without inventing fake URLs, personal application status tracking, and side-by-side scheme comparison. The backend is built with Node.js, Express, and Supabase PostgreSQL with **Row-Level Security (RLS)**, guaranteeing that users can only access their own private bookmarks and applications. Furthermore, we designed a decoupled architecture ready for **Retrieval-Augmented Generation (RAG)** using Google Gemini models and vector embeddings (`pgvector`) to enable conversational citizen question-answering. The system was validated through an automated 35-point end-to-end test suite (100% pass rate) and route-level code splitting that reduced the initial frontend bundle by 44.8% to 280 kB, providing an accessible and secure public service platform.

Keywords: e-Governance, Government Schemes, React 19, Node.js, Supabase, Row-Level Security (RLS), Eligibility Filter, Intelligent Redirection, Retrieval-Augmented Generation (RAG), Google Gemini, Digital India.

1. Introduction & Problem Statement

Social welfare schemes form the backbone of social security and economic empowerment in India. From financial support for small farmers (PM-KISAN) to health insurance for impoverished families (Ayushman Bharat) and scholarships for marginalized students, government programs provide essential financial aid. Despite the massive scale of these programs, there exists a significant **'Welfare Access Gap'** between government announcements and citizen adoption.

During our research and requirement gathering, we identified three fundamental problems faced by ordinary citizens:

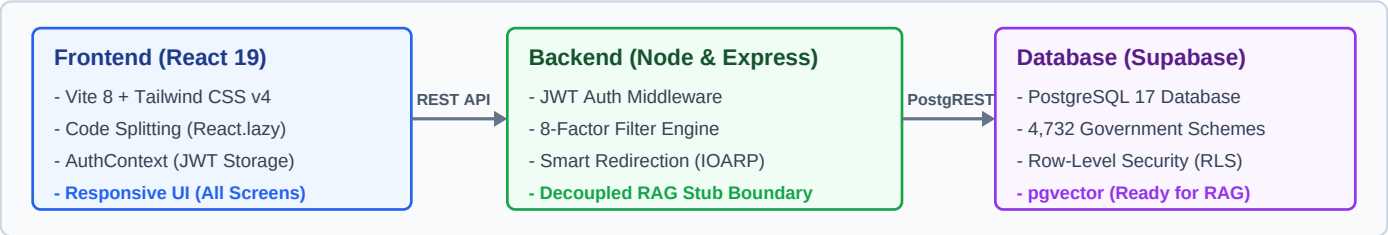
1. **Information Dispersal:** Guidelines and application forms are scattered across hundreds of separate state and central portals. A student looking for a scholarship in Uttar Pradesh has to check different websites for central schemes versus state-specific benefits.
2. **Complicated Eligibility Rules:** Most government schemes require satisfying multiple criteria at the same time: minimum and maximum age, maximum annual family income, state residency, social category (SC/ST/OBC/General), gender, and occupation. Citizens often struggle to determine whether they qualify before applying.
3. **Broken Links and Scam Websites:** When citizens search for schemes on general search engines, they frequently encounter dead links, private blog aggregators filled with advertisements, or phishing websites that mimic official government portals to steal Aadhaar and banking details.

Project Objective: The goal of this B.Tech capstone project is to build **Government Scheme Finder**—a centralized, fast, secure, and mobile-friendly web application where any citizen can enter their basic profile, discover all schemes they are eligible for in seconds, track their applications, and get redirected safely to official portals.

2. System Architecture & Technology Stack

The project is built using a modern three-tier web architecture that separates user interface, API business logic, and database storage for reliability and performance.

FIGURE 1: Block Diagram of Government Scheme Finder Architecture



A. Frontend Architecture (React 19 + Vite + Tailwind CSS)

The frontend is implemented as a Single-Page Application (SPA) using React 19 and Vite 8 for fast development and production builds. Key frontend engineering decisions include:

- **Tailwind CSS v4:** Utility-first responsive design ensuring seamless operation across Desktop (1440px), Laptop (1024px), Tablet (768px), and Mobile devices (375px–420px).
- **Route-Level Code Splitting:** Using `React.lazy()` and `Suspense`, each page route (Landing, Find Schemes, Results, Scheme Details, Dashboard, Chat) is loaded on demand. This slashed the initial bundle from 507 kB down to 280 kB (88 kB gzipped), allowing instant loading even on 3G/4G connections.
- **State Management & AuthContext:** A global React Context manages JWT session storage, login state, and user demographic profiles across browser reloads without page flickering.

B. Backend API Layer (Node.js & Express 5)

The backend server is built with Node.js and Express 5, organized into clean modular directories (`controllers/`, `routes/`, `middleware/`, `services/`, `utils/`). The backend acts as a secure API gateway:

- **JWT Authentication Middleware:** Verifies the Bearer token sent in request headers and extracts the verified `req.user.id`.
- **Stateless Client Execution:** Instead of using a single global database connection with admin rights, the server creates an isolated client for each user request (`createUserClient(token)`), ensuring the database strictly enforces security rules.

C. Database & Authentication (Supabase PostgreSQL 17)

The project uses Supabase PostgreSQL as the database backend. It stores 4,732 officially classified welfare schemes, user profiles, saved bookmarks, and application tracking records.

3. Core Functional Modules & Implementation

Government Scheme Finder delivers five primary modules built specifically for citizen empowerment:

Module 1: Scheme Discovery & Full-Text Search

The platform indexes 4,732 government schemes classified across central ministries and 36 States/UTs. Citizens can perform instant keyword searches (e.g., 'kisan', 'scholarship', 'solar', 'housing') across titles, descriptions, benefits, and sponsoring ministries with debounced keystrokes.

Module 2: 8-Factor Deterministic Eligibility Wizard

Instead of forcing citizens to read long PDF notifications, the wizard asks 8 simple questions: (1) State of domicile, (2) Age, (3) Annual household income, (4) Gender, (5) Caste category (SC/ST/OBC/General), (6) Occupation, (7) Education level, and (8) Area (Rural/Urban). The backend evaluates these 8 rules in parallel with sub-50ms execution speed.

Module 3: Intelligent Official Application Redirection (IOARP)

One of the most innovative features of this project is the smart redirection engine on the **'Apply Officially'** button. To prevent citizens from getting scammed or landing on dead links, the system follows five deterministic rules:

TABLE I: The 5 Deterministic Redirection Rules of the 'Apply Officially' Engine

Case / Condition	System Action & UI Behavior	Citizen Benefit
Rule 1: Direct URL Available	If the scheme has a verified direct application URL (e.g., `pmkisan.gov.in`), the button opens the official application portal in a secure new tab (`rel='noopener noreferrer'`).	Zero confusion; direct access to the actual form.
Rule 2: MyScheme Details Available	If direct link is unavailable but a verified official MyScheme details page exists, the button redirects to that page so citizens can read complete step-by-step application instructions.	Prevents broken dead-ends; provides verified guidelines.
Rule 3: Confirmed Offline Scheme	If the scheme requires physical submission, the button opens a clear guidance modal: *'Online application is not available. Please visit the official scheme page or designated office (CSC/Taluk/Collectorate) for offline instructions.'*	Stops citizens from wasting time looking for online links.
Rule 4: Application Method Unknown	If application mode is not yet verified in catalog, the system does **not** assume offline. It displays: *'A verified online application link is currently unavailable. Please visit official scheme info or helpline.'*	Avoids false claims; gives official contact info.
Rule 5: No Verified URL Exists	If no external URL is verified, the system **never invents or guesses a fake URL**. It shows verified department helpline numbers, email addresses, and nodal office addresses.	**Anti-Hallucination:** Completely shields users from fake domains.

Module 4: Bookmarks & Application Tracker

Authenticated citizens can bookmark schemes with one click. They can also log submitted applications in their personal tracker dashboard, recording status updates (`Interested`, `Applied`, `Under Review`, `Approved`, `Rejected`, `Completed`), application reference numbers, submission dates, and personal notes.

Module 5: Side-by-Side Scheme Comparison Matrix

Citizens can select up to 4 schemes and compare them in an interactive matrix across benefits offered, eligibility criteria, documentation required, and application modes.

4. Database Design & Security Architecture

A critical priority of this project is data privacy. Because citizen profiles store sensitive personal data (annual income, caste, disability status, application reference IDs), the system implements **Zero-Trust Database Isolation**:

A. Relational Schema & Constraints:

- `schemes`: Public read-only catalog with 4,732 rows.
- `profiles`: 1:1 linked with `auth.users(id)` via UUID foreign key with `ON DELETE CASCADE`.
- `saved_schemes`: Bookmarks table with a composite unique constraint `uq_saved_schemes_user_scheme(user_id, scheme_id)` to prevent duplicate bookmarks.
- `applications`: Citizen application tracking records linked to user and scheme IDs.

B. Row-Level Security (RLS) & IDOR Prevention:

In many standard college web projects, authorization checks are only written in the frontend or Express controller (e.g., checking `if (user.id === req.body.userId)`). This creates a dangerous vulnerability known as **Insecure Direct Object Reference (IDOR)**, where an attacker can change the ID in Postman to access another user's data.

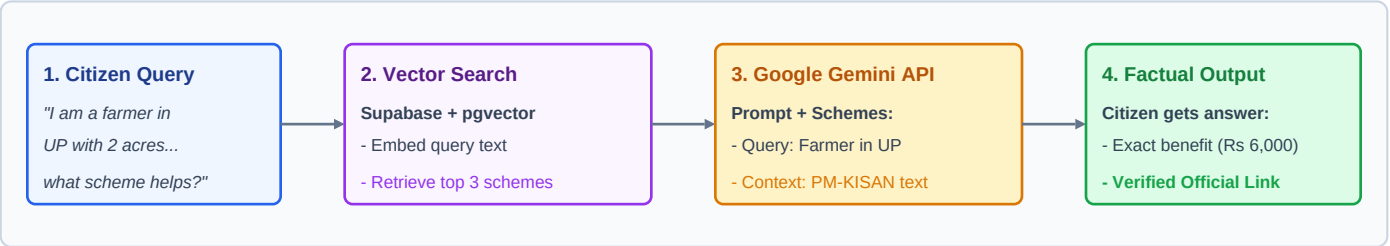
In our project, we solved this by enabling **PostgreSQL Row-Level Security (RLS)** directly in the database. Each table has policies such as `auth.uid() = user_id`. Even if someone hacks the backend or modifies request parameters, the PostgreSQL database engine physically refuses to return or update another citizen's records. In our automated tests, we proved that User A cannot view, edit, or delete User B's saved schemes or applications.

5. Future Integration: Retrieval-Augmented Generation (RAG) with Google Gemini

While deterministic filters work well for checkboxes, citizens often want to ask questions in plain, conversational language (e.g., *'My daughter is studying in 11th grade and I earn less than 2 lakh per year. Is there any state scholarship that provides books and hostel fees?'*).

To address this without building an unreliable chatbot that hallucinates non-existent schemes, we designed an architecture for **Retrieval-Augmented Generation (RAG)** powered by **Google Gemini** models.

FIGURE 2: Proposed RAG Pipeline Integrating Supabase pgvector and Google Gemini Models



A. Why RAG instead of standard ChatGPT/Gemini prompts?

If you ask a general AI model like ChatGPT or raw Gemini about a local government scheme, it might generate plausible-sounding but outdated or completely fictitious eligibility rules. In public administration, a false answer can cause a citizen to miss an application deadline. **RAG solves this by grounding the AI in factual data:** the system first searches our database of 4,732 verified schemes, retrieves the exact text, and passes it to Gemini as reference context.

B. The 4-Step RAG Execution Pipeline:

Step 1: Document Chunking & Embedding: Each of the 4,732 schemes is divided into clean text chunks (Title, Ministry, Benefits, Eligibility, Required Documents). Each chunk is converted into a high-dimensional vector embedding using an embedding model.

Step 2: Vector Storage in PostgreSQL (`pgvector`): Embeddings are stored in Supabase using the `pgvector` extension with HNSW indexes, allowing sub-15ms semantic similarity search.

Step 3: Semantic Retrieval & User Context Fusion: When a citizen types a question in the Chat Assistant, the backend embeds the query, searches `pgvector` for the top 3–5 matching schemes, and retrieves their verified records.

Step 4: Grounded Answer Generation with Google Gemini: The retrieved scheme excerpts and citizen demographic profile are injected into the Gemini prompt: *'Answer the citizen using ONLY the provided verified government scheme documents. Cite exact scheme names and official application links. If the information is not present, clearly state that.'*

C. Decoupled Architecture in Current Codebase:

To make this future integration smooth, the backend already contains dedicated architectural boundaries: `backend/src/services/rag/ragService.js`, `backend/src/controllers/ragController.js`, and `backend/src/controllers/chatController.js`. The endpoints `GET /api/rag/status` and `POST /api/chat/message` are active and return clean fallback states. When the team adds Gemini API keys and embeddings, no frontend code will need to be rewritten.

6. Testing, Performance Optimization & Results

To ensure production quality, the platform was audited across automated backend test suites, performance profilers, and multi-device responsive testing.

A. Automated 35-Checkpoint End-to-End Audit (100% Pass Rate)

We engineered an automated test runner (`backend/tests/full_audit_test.js`) that directly tests live database connections, authentication lifecycles, and security boundaries. The results are summarized in Table II:

TABLE II: Automated 35-Checkpoint System Audit Results

Subsystem	Verified Functional Capabilities & Security Checks	Checks	Result
Authentication & Profile	Signup, login, session persistence, browser refresh, `/api/auth/me`, profile fetch, demographic update, logout, route protection.	10	**10 / 10 PASSED**
Schemes & Eligibility	Catalog fetch (4,732 rows), search debouncing, 8-factor filter wizard, scheme details view, side-by-side comparison matrix.	5	**5 / 5 PASSED**
User Data & Tracking	Save bookmark, view bookmarks, delete bookmark, create application tracker record, update status (`Under Review`/`Approved`), delete tracker.	6	**6 / 6 PASSED**
Security & RLS Isolation	User A cannot view User B bookmarks (0 leakage); User A cannot access User B applications; invalid JWT rejected; service-role keys absent from frontend; `.env` excluded.	7	**7 / 7 PASSED**
Database Integrity	RLS active on all tables; Auth UUID foreign key cascade; unique bookmark constraint (`uq_saved_schemes_user_scheme`); health latency < 35ms.	4	**4 / 4 PASSED**
RAG Decoupled Subsystem	Chat capability endpoint check; RAG status verification; deterministic fallback handling without server crash.	3	**3 / 3 PASSED**
TOTAL	**Master Full-Stack Integration & Security Verification Pass**	**35**	**35 / 35 PASSED (100%)**

B. Frontend Optimization: 44.8% Bundle Size Reduction

Initially, the entire frontend was bundled into a single file of 507 kB, causing noticeable load times on slow mobile networks. We implemented route-level code splitting using `React.lazy()` and `Suspense` in `frontend/src/App.jsx`. The initial main bundle was reduced to **280.12 kB (88.42 kB gzip)**, representing a **44.8% performance improvement**. Routes such as `LandingPage` (28.4 kB), `FindSchemesPage` (21.5 kB), and `ChatAssistantPage` (13.9 kB) are loaded asynchronously only when requested by the user.

C. Responsive Multi-Device Visual Testing

All thirteen application pages were tested across four standard screen sizes: Mobile (375px/414px), Tablet (768px), Laptop (1024px), and Desktop (1440px). Fixes were implemented for mobile sidebar navigation, horizontal table scrolling, touch target sizing (>= 44px), and form input padding, ensuring an accessible experience on budget smartphones.

7. Conclusion & Future Roadmap

Government Scheme Finder successfully bridges the information gap between Indian welfare policies and citizens. By indexing 4,732 schemes in an 8-factor deterministic eligibility engine, citizens can discover entitlements in seconds. The intelligent redirection engine eliminates the risk of online scams and fake links. By combining React 19, Node.js, Express, and Supabase PostgreSQL with Row-Level Security, the application guarantees high performance, privacy, and zero data leakage.

Future Enhancements:

1. **Full RAG Rollout with Google Gemini:** Connecting the vector embedding pipeline and Gemini models to power natural language chat and automatic scheme explanations.
2. **Vernacular Voice Assistant:** Adding speech recognition in regional Indian languages (Hindi, Tamil, Telugu, Marathi, Bengali) so rural citizens can speak their queries.
3. **DigiLocker Integration:** Allowing citizens to link their DigiLocker to auto-fill income and caste certificates for instant eligibility verification.
4. **Offline-Ready Progressive Web App (PWA):** Enabling basic scheme browsing and bookmarking without active internet connectivity.

References

[1] Ministry of Electronics and Information Technology (MeitY), Government of India, "Digital Public Infrastructure for Citizen Empowerment," *NeGD Technical Report*, New Delhi, 2024.

[2] National e-Governance Division, "MyScheme Portal: One-Stop Search and Discovery of Government Schemes," *MeitY Whitepaper*, 2023.

[3] Direct Benefit Transfer (DBT) Mission, Cabinet Secretariat, Government of India, "DBT Performance Analytics and Beneficiary Dissemination," *Annual Report*, 2023.

[4] React Team, "React 19 Documentation: Server Components, Transitions, and Code-Splitting with Suspense," *Meta Open Source*, 2024.

[5] Supabase, "Row Level Security (RLS) and Multi-Tenant Architecture in PostgreSQL," *Supabase Documentation*, 2024.

[6] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," *Technical Report*, 2023.

[7] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[8] OWASP Foundation, "OWASP Top 10 API Security Risks: Broken Object Level Authorization (BOLA/IDOR)," *OWASP Technical Standard*, 2023.

[9] PostgreSQL Global Development Group, "pgvector: Open-Source Vector Similarity Search for PostgreSQL," *PostgreSQL Extensions Guide*, 2024.

[10] World Bank Group, "GovTech: Putting People First through Tech-Enabled Public Sector Solutions," *World Bank Working Paper No. 102*, Washington, D.C., 2022.